PRÁCTICA 1

1. Ejercicio 1 a (malla.c) 4
1.1. Enunciado del Ejercicio 4
Ejemplo de Salida Esperada 5
1.2. Identificación y Justificación de Tareas 5
1.2.1. Recepción y Validación de Argumentos 5
1.2.2. Generación Concurrente de la Malla de Procesos 5
1.2.3. Sincronización y Notificación del Nodo Final (pxy) 5
1.2.4. Muestra de la Jerarquía (pstree) mediante Proceso Hijo Efímero 6
1.2.5. Gestión de Estado Global y Configuración de Señales 6
1.2.6. Destrucción Controlada y Propagación en Cascada 6
1.3 Diagrama de flujo 7
1.4. Análisis Detallado del Código Fuente 7
1.4.1. Módulo Principal (malla.c) 8
Muestra del árbol y flujo principal (main): 8
1.4.2. Módulo de Creación Horizontal (horizontal.c) 9
1.4.3. Módulo de Creación Vertical (vertical.c) 10
Lógica de construcción en profundidad y propagación: 10
1.4.4. Módulo de Señales y Control (pfinal.c) 11
2. Ejercicio 1 b (ejec.c) 12
2.1. Enunciado del Ejercicio 12
Estructura del Árbol de Procesos y Resultado Esperado 12
2.2. Identificación y Justificación de Tareas 12
2.2.1. Recepción y Validación de Argumentos 13
2.2.2. Generación Jerárquica del Árbol de Procesos 13
2.2.3. Temporización Asíncrona mediante Alarmas (Proceso Z) 13
2.2.4. Notificación y Muestra de la Estructura (pstree) 13
2.2.5. Coordinación Interproceso y Compartición de Estado Global 13
2.2.6. Destrucción Ordenada y Cascada de Espera (wait) 14
2.3 Diagrama de flujo 15
2.4. Análisis Detallado del Código Fuente 16
2.4.1. Módulo Raíz (ejec.c) 16
Estructura del flujo principal (main): 16
2.4.2. Módulo de Intermediación y Visualización (A.c) 17
Rutina de muestra de árbol e intermediación: 17
2.4.3. Módulo de Distribución de Ramas (B.c) 18
Lógica de iteración y propagación: 19
2.4.4. Módulos Hojas de Espera (X.c y Y.c) 19
Estructura típica de un proceso hoja (X.c / Y.c): 20
2.4.5. Módulo Temporizador (Z.c) 20

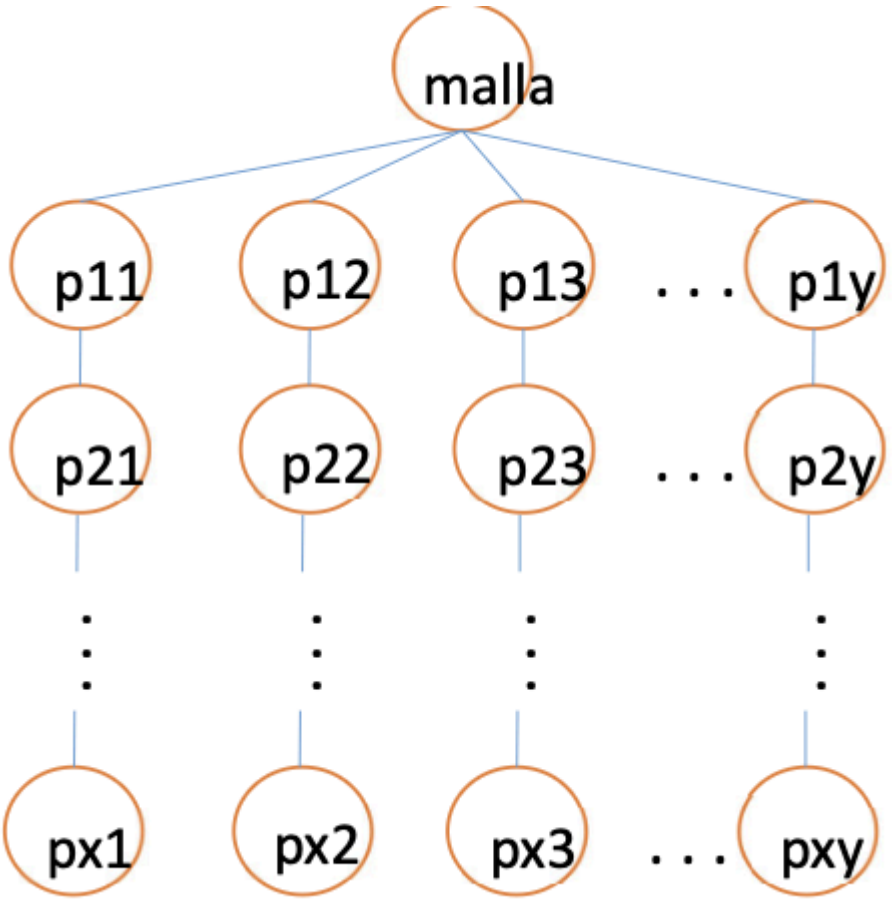
Implementación del temporizador y señalización: 20
3. Ejercicio 2 (hacha.c) 21
3.1. Enunciado del Ejercicio 21
Ejemplo de Uso y Resultado Esperado 22
3.2. Identificación y Justificación de Tareas 22
3.2.1. Validación de Argumentos e Inspección del Fichero (struct stat) 22
3.2.2. Cálculo de Fragmentación y Apertura en Modo Lectura 22
3.2.3. Creación Concurrente de Procesos y Tuberías Anónimas (IPC) 23
3.2.4. Transmisión de Datos por Tuberías y Apertura de Salidas (creat) 23
3.2.5. Sincronización y Fin de Fichero (EOF) 23
3.3 Diagrama de flujo 24
3.4. Análisis Detallado del Código Fuente 25
3.4.1. Módulo de Inspección, Apertura y Cálculo 25
Inspección del tamaño del archivo mediante struct stat: 25
3.4.2. Lógica del Proceso Consumidor / Hijo (ejecutar_hijo) 26
3.4.3. Lógica del Proceso Productor / Padre (ejecutar_padre) 27
3.4.4. Orquestación Concurrente y Sincronización Final 28
4. Ejercicio 3 (hijos.c) 30
4.1. Enunciado del Ejercicio 30
4.2. Identificación y Justificación de Tareas 30
4.2.1. Validación de Parámetros de Entrada (parse) 30
4.2.2. Gestión Centralizada de Memoria Compartida (struct MemoriaCompartida) 31
4.2.3. Creación Recursiva de la Cadena Vertical (crearVertical) 31
4.2.4. Generación Horizontal y Modularización de Subhijos (crearHorizontal) 31
4.2.5. Ejecución de pstree y Destrucción en Cascada 32
4.3 Diagrama de flujo 32
4.4. Análisis Detallado del Código Fuente 33
4.4.1. Módulo Principal y Gestión IPC (main) 33
4.4.2. Módulo de Creación Vertical Recursiva (crearVertical) 34
4.4.3. Módulo de Subhijos Horizontales (crearHorizontal) 35
5. Recursos utilizados 37
5.1 Mermaid.live 37
5.2 Github 37

1. Ejercicio 1 a (malla.c)

1.1. Enunciado del Ejercicio

Realiza un programa llamado malla.c que reciba dos argumentos por línea de comandos, x e y, representando respectivamente el número de filas y columnas de una estructura de procesos en forma de malla.

El objetivo principal del programa es la creación correcta de dicha topología de procesos y la posterior verificación de su estructura en el sistema operativo mediante la ejecución de la orden pstree -c lanzada por el proceso superpadre.



Ejemplo de Salida Esperada

Sintaxis de invocación por terminal:

$ ./malla 3 2

Salida esperada (volcado de pstree -c ejecutado tras la creación de la malla):

```
malla─┬─malla───malla───malla
      ├─malla───malla───malla
      └─pstree
```

1.2. Identificación y Justificación de Tareas

A partir del análisis detallado del enunciado, se identifican y justifican las siguientes tareas técnicas de diseño e implementación:

1.2.1. Recepción y Validación de Argumentos

El programa requiere verificar mediante parse() que el usuario introduzca exactamente dos argumentos adicionales (argc == 3). Además, se comprueba que los parámetros x e y correspondan a valores enteros estrictamente positivos, impidiendo la configuración de dimensiones nulas o negativas que invaliden la formación de la malla.

1.2.2. Generación Concurrente de la Malla de Procesos

La construcción de la malla de dimensiones $x \times y$ se descompone de forma modular:

- **Dimensión Horizontal:** La función crearHorizontal() en horizontal.c genera y procesos hijos directos del superpadre (uno por cada columna). Cada uno actúa como cabeza de su respectiva columna.

- **Dimensión Vertical:** Cada cabeza de columna invoca crearVertical() en vertical.c, encadenando $x - 1$llamadas a fork() para construir la profundidad de la columna. Cada proceso de la columna conoce únicamente a su hijo directo inferior.

1.2.3. Sincronización y Notificación del Nodo Final (p_{xy})

Para que el superpadre sepa exactamente cuándo la malla se ha completado en su totalidad, se delega en el último proceso creado (esquina inferior derecha, posición $x - 1, y - 1$) la responsabilidad de notificar al superpadre (pidPadreOriginal). En codigo_ultimo_hijo(), este proceso programa un temporizador con alarm(1)y se

bloquea en pause(). Al vencer la alarma, su manejador envía la señal SIGUSR1 al superpadre.

1.2.4. Muestra de la Jerarquía (pstree) mediante Proceso Hijo Efímero

Al recibir la señal SIGUSR1, el superpadre despierta de su reposo (pause()). Para evitar reemplazar su propia imagen en memoria (lo que le impediría coordinar la posterior destrucción de los procesos), la función llamadaPstree() crea un proceso hijo efímero con fork(), el cual ejecuta execlp("pstree", "pstree", "-c", ...). El superpadre aguarda la finalización de este hijo mediante wait() antes de proceder con el apagado.

1.2.5. Gestión de Estado Global y Configuración de Señales

Para permitir la interacción entre los distintos ficheros compilados por separado (malla.c, horizontal.c, vertical.c, pfinal.c), se define la variable global pidPadreOriginal, exportada mediante vinculación externa (extern). Además, se registra el manejador asíncrono despertar() mediante signal() para garantizar que los procesos puedan reanudar su ejecución tras recibir señales sin finalizar prematuramente.

1.2.6. Destrucción Controlada y Propagación en Cascada

Para evitar el desperdicio de recursos del sistema operativo o la generación de procesos huérfanos/zombis, se implementa un sistema de envio de señales en cascada de arriba hacia abajo que desencadena una destrucción ordenada de abajo hacia arriab

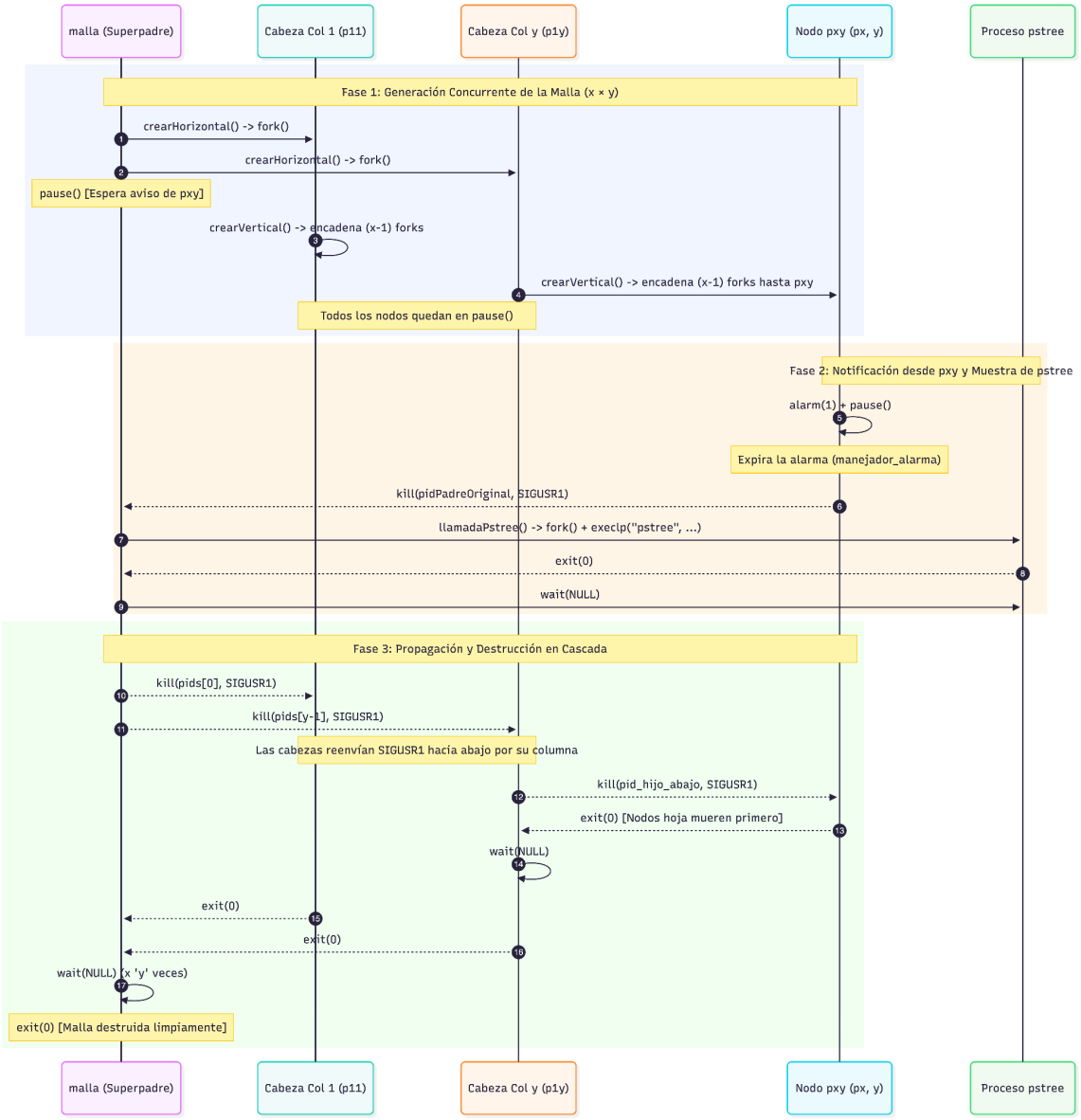
1. El superpadre envía la señal SIGUSR1 a cada una de las cabezas de columna almacenadas en su vector local pids[].

2. Cada nodo de la columna despierta, reenvía SIGUSR1 a su hijo directo inferior (pid_hijo_abajo) mediante kill() y se bloquea en wait(NULL).

3. Los nodos de la última fila, al no tener descendientes directos, finalizan inmediatamente con exit(0).

4. La recogida de muertes asciende en cascada por cada columna hasta que el superpadre recolecta a las cabezas de columna mediante un bucle de wait().

1.3 Diagrama de flujo



1.4. Análisis Detallado del Código Fuente

A continuación se detalla la responsabilidad técnica, la gestión del estado global compartido y las llamadas al sistema utilizadas en cada uno de los módulos.

1.4.1. Módulo Principal (malla.c)

Este módulo inicializa la aplicación, valida las dimensiones requeridas, guarda el PID del superpadre e invoca la muestra del árbol y la fase de destrucción.

- **Variables Globales Exportadas:**
    - pid_t pidPadreOriginal: almacena el PID del proceso superpadre y se exporta mediante extern a los módulos pfinal.c y vertical.c.
- **Llamadas al Sistema Utilizadas:** getpid(), fork(), execlp(), pause(), kill(), wait(), exit().

Muestra del árbol y flujo principal (main):

```
pid_t pidPadreOriginal; // Variable global accesible desde otros
ficheros

void llamadaPstree(pid_t pidpadreOriginal) {
    if (fork() == 0) {
        char pid_str[16];
        sprintf(pid_str, "%d", pidpadreOriginal);
        execlp("pstree", "pstree", "-c", pid_str, NULL);
        perror("Error en execlp");
        exit(1);
    }
}
```

```
int main(int argc, char *argv[]) {
    parse(argc, argv);
    int filas = atoi(argv[1]);
    int columnas = atoi(argv[2]);
    pidPadreOriginal = getpid();

    configurar_senyales();

    pid_t pids[columnas]; // Almacena PIDs de las cabezas de
columna
    crearHorizontal(filas, columnas, pids);

    pause(); // Espera a la finalización de la malla (notificación
del último hijo)
```

```
    llamadaPstree(pidPadreOriginal);
    wait(NULL); // Espera a que concluya el proceso que ejecutó
pstree

    // Notificación de terminación a la primera fila
    for (int i = 0; i < columnas; i++) kill(pids[i], SIGUSR1);
    for (int i = 0; i < columnas; i++) wait(NULL); // Recolección
final

    return 0;
}
```

1.4.2. Módulo de Creación Horizontal (horizontal.c)

Encargado de generar los procesos correspondientes a la primera fila de la malla.

- **Comportamiento Técnico:** Recibe la dimensión horizontal y, instanciando cada cabeza de columna mediante fork() y guardando los PIDs resultantes en el vector pids[] para permitir al superpadre notificarles en la fase de apagado. Cada hijo invoca a crearVertical().

- **Llamadas al Sistema Utilizadas:** fork(), exit().

```
void crearHorizontal(int x, int y, pid_t pids[]) {
    for (int col = 0; col < y; col++) {
        pid_t pid = fork();

        if (pid == 0) { // Proceso Hijo (Cabeza de Columna)
            crearVertical(col, x, y);
            exit(0);
        } else if (pid > 0) {
            pids[col] = pid; // Guardado de PID en el vector del
superpadre
        } else {
            perror("Error en fork");
            exit(1);
        }
    }
}
```

1.4.3. Módulo de Creación Vertical (vertical.c)

Construye la profundidad vertical de cada columna y gestiona la notificación del nodo situado en la esquina inferior derecha.

- **Variables Importadas:**
    - extern pid_t pidPadreOriginal: referencia a la variable definida en malla.c para poder notificar la finalización.
- **Llamadas al Sistema Utilizadas:** fork(), signal(), alarm(), pause(), kill(), wait(), exit().

Lógica de construcción en profundidad y propagación:

```
void codigo_ultimo_hijo(void) {
    signal(SIGALRM, manejador_alarma);
    alarm(1);
    pause(); // Despierta tras 1s y envía SIGUSR1 al superpadre
}

void crearVertical(int col, int x, int y) {
    int fila = 0;
    pid_t pid_hijo_abajo = -1; // Conserva el PID del hijo directo
inferior

    for (int f = 1; f < x; f++) {
        pid_t pid = fork();
        if (pid > 0) {
            pid_hijo_abajo = pid; // El padre guarda a su hijo
directo y sale
            break;
        }
        fila = f;
    }

    // Identificación del nodo esquina inferior derecha (pxy)
    if (col == (y - 1) && fila == (x - 1)) {
        codigo_ultimo_hijo();
    }

    signal(SIGUSR1, despertar);
    pause(); // Bloqueo a la espera de la orden de terminación
```

```
    // Propagación descendente de la orden de destrucción
    if (pid_hijo_abajo > 0) {
        kill(pid_hijo_abajo, SIGUSR1);
        wait(NULL); // Garantiza no morir antes que el hijo
inferior
    }
    exit(0);
}
```

1.4.4. Módulo de Señales y Control (pfinal.c)

Agrupa los manejadores de señales compartidos por toda la aplicación.

- **Variables Importadas:**
    - extern pid_t pidPadreOriginal: destinatario de la señal emitida al saltar la alarma.
- **Llamadas al Sistema Utilizadas:** signal(), kill().

```
extern pid_t pidPadreOriginal;

void despertar(int sig) {} // Cuerpo vacío para interrumpir
pause() de forma segura

void manejador_alarma(int sig) {
    kill(pidPadreOriginal, SIGUSR1); // Notifica la finalización
de la malla
}

void configurar_senyales(void) {
    signal(SIGUSR1, despertar);
}
```

2. Ejercicio 1 b (ejec.c)

2.1. Enunciado del Ejercicio

Realiza un programa llamado **ejec.c** que reciba un argumento. El programa tendrá que generar el árbol de procesos que se indica y llevar a cabo la funcionalidad que se describe a continuación. El proceso Z, transcurridos los segundos indicados por el argumento, enviará una señal al proceso A. El proceso A, al recibir la señal, ejecutará el comando “pstree”. El proceso Z no puede utilizar el comando “sleep”, por lo que el proceso Z debe planificarse una alarma con los segundos indicados por el argumento. Se deberá controlar la correcta destrucción del árbol (los padres no pueden morir antes que los hijos). **(2 puntos)**

Estructura del Árbol de Procesos y Resultado Esperado

La jerarquía a construir responde a la siguiente genealogía de procesos:

Salida por terminal esperada (ejemplo para ./ejec 15):

Soy el proceso ejec: mi pid es 751
Soy el proceso A: mi pid es 752. Mi padre es 751
Soy el proceso B: mi pid es 753. Mi padre es 752. Mi abuelo es 751
Soy el proceso X: mi pid es 754. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
Soy el proceso Y: mi pid es 755. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
Soy el proceso Z: mi pid es 756. Mi padre es 753. Mi abuelo es 752. Mi bisabuelo es 751
<Salida de pstree -c tras 15 segundos>
Soy Z (756) y muero
Soy Y (755) y muero
Soy X (754) y muero
Soy B (753) y muero
Soy A (752) y muero
Soy ejec (751) y muero

2.2. Identificación y Justificación de Tareas

A partir de los requisitos explícitos e implícitos enunciados, se identifican y justifican las siguientes tareas de diseño e implementación:

2.2.1. Recepción y Validación de Argumentos

El programa requiere la introducción por línea de comandos de la duración en segundos de la espera de **Z**. La función parse() en ejec.c asume la tarea de comprobar que argc == 2 y que el parámetro sea un número entero estrictamente positivo. Se justifica esta validación previa para prevenir comportamientos indeterminados o llamadas a alarm() con valores nulos o negativos.

2.2.2. Generación Jerárquica del Árbol de Procesos

Para construir la jerarquía requerida (ejec → A → B → {X, Y, Z}), el sistema se modulariza en ficheros independientes por proceso (ejec.c, A.c, B.c, X.c, Y.c, Z.c).

- ejec engendra a A.
- A engendra a B.
- B utiliza un bucle con un condicional switch para bifurcar y crear consecutivamente a sus tres procesos hijos X, Y y Z.

Esta estructura evita acoplamientos y permite que cada proceso ejecute su propio código especializado tras el fork().

2.2.3. Temporización Asíncrona mediante Alarmas (Proceso Z)

El enunciado prohíbe explícitamente el uso de bloqueos activos o la función sleep(). Se justifica así el uso de alarm(segundos) combinado con la llamada al sistema pause(). El proceso **Z** delega en el kernel del sistema operativo la cuenta atrás, permaneciendo bloqueado en estado de reposo de bajo consumo hasta la llegada de la señal SIGALRM.

2.2.4. Notificación y Muestra de la Estructura (pstree)

Al expirar la alarma, el proceso **Z** notifica al proceso **A** mediante la señal SIGUSR1. Para cumplir con el requisito de mostrar el árbol mediante pstree -c sin reemplazar la imagen en memoria del proceso **A** (lo que impediría continuar con la fase de destrucción), **A** realiza un fork() adicional para delegar la sustitución del código mediante execlp("pstree", ...) a un hijo efímero, esperando su finalización mediante wait().

2.2.5. Coordinación Interproceso y Compartición de Estado Global

Para que los procesos hoja (X, Y, Z) y descendientes intermedios puedan imprimir por pantalla no solo su PID y el de su padre (getppid()), sino también los identificadores de sus ancestros (abuelo y bisabuelo), se justifica la definición de variables globales compartidas mediante vinculación externa (extern):

- pidSuperPadre: almacena el PID de la raíz ejec.
- pid_A: almacena el PID del proceso A.

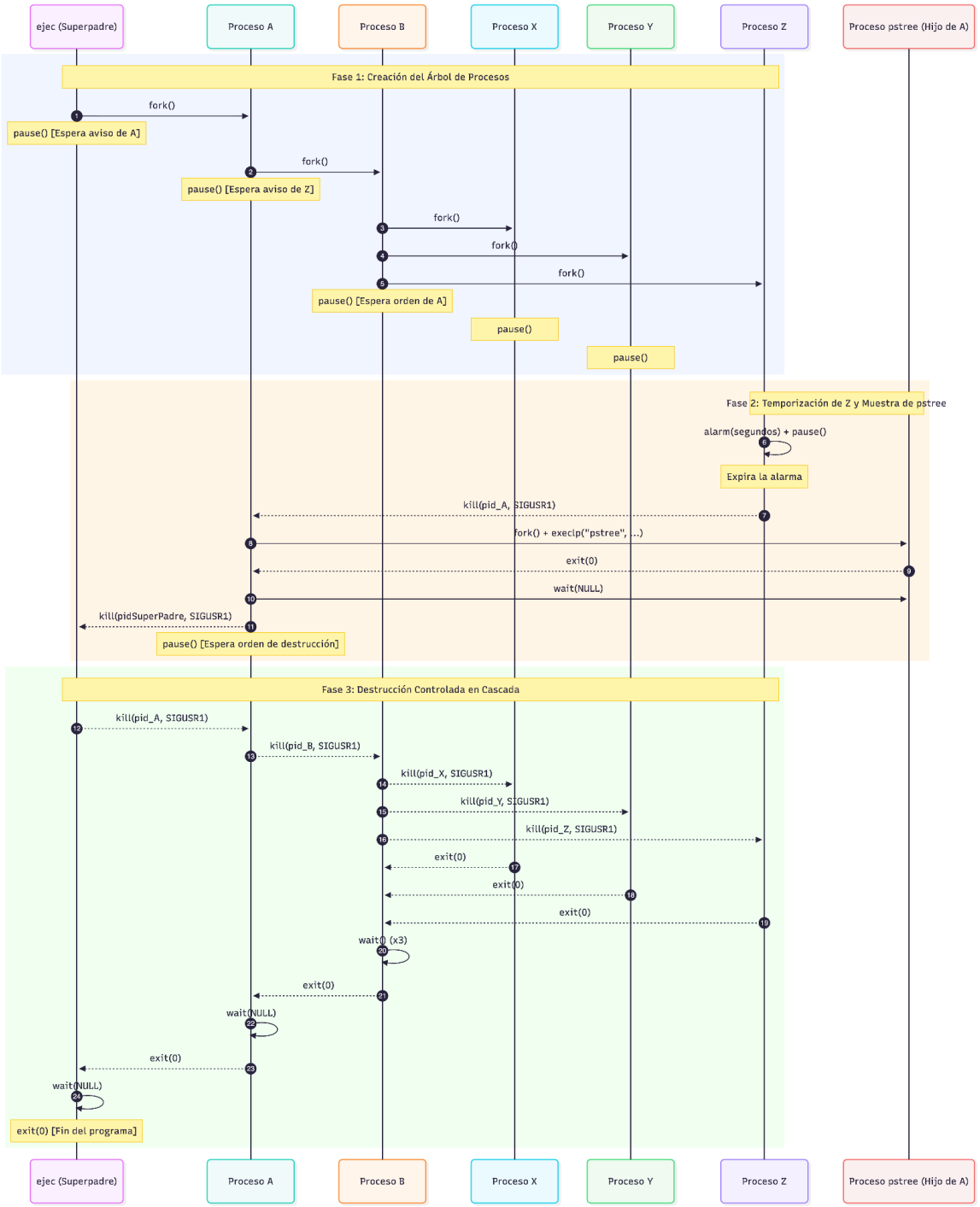
Asimismo, el manejador de señales reentrante y de cuerpo vacío despertar() se comparte globalmente para interrumpir llamadas a pause() de forma segura.

2.2.6. Destrucción Ordenada y Cascada de Espera (wait)

Para garantizar que ningún padre muera antes que sus hijos, se establece un protocolo de sincronización bidireccional mediante señales y llamadas wait():

1. Una vez ejecutado pstree, **A** notifica a ejec mediante SIGUSR1 y queda en pause().
2. ejec despierta y ordena la destrucción enviando SIGUSR1 a **A**, bloqueándose en wait().
3. **A** reenvía la orden a **B** y se bloquea en wait().
4. **B** propaga la orden enviando SIGUSR1 a X, Y y Z mediante su vector de PIDs, y ejecuta un bucle de tres llamadas a wait().
5. Los procesos hoja despiertan, muestran su mensaje de terminación y ejecutan exit(0).
6. La cascada de recolección asciende progresivamente hasta que el superpadre ejec recoge a **A** y finaliza el programa.

2.3 Diagrama de flujo



2.4. Análisis Detallado del Código Fuente

A continuación se detalla la responsabilidad técnica, la gestión del estado global y las llamadas al sistema empleadas en cada uno de los módulos que forman el programa. Se ha simplificado el código omitiendo fragmentos dedicados a la gestión de errores en fork o similares para enfocar esta sección hacia la comprensión teórica de cada uno de los fragmentos, no su implementación concreta.

2.4.1. Módulo Raíz (ejec.c)

Este módulo implementa el proceso superpadre. Su función es inicializar la variable global pidSuperPadre, validar los parámetros de entrada y actuar como el punto final de control de la destrucción.

- **Variables Globales y Exportadas:**
  - pid_t pidSuperPadre: almacena el PID del proceso raíz ejec para que sea accesible desde el resto de módulos mediante extern pid_t pidSuperPadre.
- **Manejadores de Señal:**
  - void despertar(int sig): función con cuerpo vacío cuya única tarea es interrumpir las llamadas bloqueantes pause() sin abortar el proceso.
- **Llamadas al Sistema Utilizadas:** getpid(), fork(), signal(), pause(), kill(), wait(), exit().

Estructura del flujo principal (main):

```
// Definición de la variable global compartida
pid_t pidSuperPadre;

void despertar(int sig) {}

int main(int argc, char *argv[]) {
    parse(argc, argv); // Validación de argumentos
    pidSuperPadre = getpid();
    printf("Soy el proceso ejec: mi pid es %d\n", pidSuperPadre);

    pid_t pid_A = fork();
    if (pid_A == 0) {
        codigo_A(atoi(argv[1])); // Transición al proceso A
    } else {
```

```
        // El superpadre espera la confirmación de muestra de
pstree
        signal(SIGUSR1, despertar);
        pause();

        // Inicia la destrucción ordenada en cascada
        kill(pid_A, SIGUSR1);
        wait(NULL); // Garantiza no morir antes que A
        printf("Soy ejec (%d) y muero\n", getpid());
        exit(0);
    }
}
```

2.4.2. Módulo de Intermediación y Visualización (A.c)

Representa el proceso **A**. Asume la responsabilidad de registrar su PID en la variable global compartida pid_A, engendrar a **B**, coordinar la muestra del árbol de procesos y canalizar las órdenes de destrucción.

- **Variables Globales:**
    - pid_t pid_A: almacena el PID de **A** y se comparte externamente con los procesos hoja (X, Y, Z).
    - extern pid_t pidSuperPadre: referencia a la variable global definida en ejec.c.
- **Llamadas al Sistema Utilizadas:** getpid(), getppid(), fork(), signal(), pause(), execlp(), kill(), wait(), exit().

Rutina de muestra de árbol e intermediación:

```
void llamadaPstree(pid_t pidSuperPadre) {
    if (fork() == 0) {
        char pid_str[16];
        sprintf(pid_str, "%d", pidSuperPadre);
        execlp("pstree", "pstree", "-c", pid_str, NULL);
        exit(1);
    }
}

void codigo_A(int segundos) {
```

```
    pid_A = getpid();
    printf("Soy el proceso A: mi pid es %d. Mi padre es %d\n",
pid_A, getppid());

    pid_t pid_B = fork();
    if (pid_B == 0) {
        codigo_B(segundos);
    } else {
        // Espera la señal de Z tras la alarma
        signal(SIGUSR1, despertar);
        pause();

        // Muestra el árbol con un hijo efímero
        llamadaPstree(pidSuperPadre);
        wait(NULL);

        // Avisa a ejec y espera la orden de destrucción
        kill(pidSuperPadre, SIGUSR1);
        pause();

        // Propaga la destrucción a B y recoge su muerte
        kill(pid_B, SIGUSR1);
        wait(NULL);
        printf("Soy A (%d) y muero\n", getpid());
        exit(0);
    }
}
```

2.4.3. Módulo de Distribución de Ramas (B.c)

El proceso **B** actúa como nodo distribuidor. Se encarga de engendrar a la totalidad de los procesos hoja (X, Y, Z) iterando en un bucle simple y guardando los PIDs resultantes en un vector local pids[3].

- **Estructuras de Datos y Variables:**
    - pid_t pids[3]: array local para almacenar los identificadores de los hijos X (índice 0), Y (índice 1) y Z(índice 2).
    - extern pid_t pidSuperPadre: permite imprimir la ascendencia completa del proceso.

- **Llamadas al Sistema Utilizadas:** getpid(), getppid(), fork(), signal(), pause(), kill(), wait(), exit().

Lógica de iteración y propagación:

```
void codigo_B(int segundos) {
    pid_t pids[3];
    printf("Soy el proceso B: mi pid es %d. Mi padre es %d. Mi
abuelo es %d\n",
           getpid(), getppid(), pidSuperPadre);

    for (int i = 0; i < 3; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            switch(i) {
                case 0: codigo_X(); break;
                case 1: codigo_Y(); break;
                case 2: codigo_Z(segundos); break;
            }
        } else {
            pids[i] = pid; // Guarda los PIDs para la propagación
        }
    }

    signal(SIGUSR1, despertar);
    pause(); // Espera la orden de terminación de A

    // Notificación en abanico a los tres procesos hoja
    for (int i = 0; i < 3; i++) kill(pids[i], SIGUSR1);
    for (int i = 0; i < 3; i++) wait(NULL); // Recolección de
hijos

    printf("Soy B (%d), y muero\n", getpid());
    exit(0);
}
```

2.4.4. Módulos Hojas de Espera (X.c y Y.c)

Los procesos **X** e **Y** representan nodos terminales (hojas) del árbol. Su comportamiento es pasivo y simétrico: tras su nacimiento e identificación de ancestros mediante el uso de pid_A y pidSuperPadre, quedan bloqueados en pause() a la espera de la orden de destrucción enviada por **B**.

- **Variables Importadas:**
    - extern pid_t pid_A: identificador del abuelo.
    - extern pid_t pidSuperPadre: identificador del bisabuelo.
- **Llamadas al Sistema Utilizadas:** getpid(), getppid(), signal(), pause(), exit().

Estructura típica de un proceso hoja (X.c / Y.c):

```
void codigo_X() {
    printf("Soy el proceso X: mi pid es %d. Mi padre es %d. Mi
abuelo es %d. Mi bisabuelo es %d\n",
           getpid(), getppid(), pid_A, pidSuperPadre);

    signal(SIGUSR1, despertar);
    pause(); // Queda a la espera de la orden de B

    printf("Soy X (%d), y muero\n", getpid());
    exit(0);
}
```

2.4.5. Módulo Temporizador (Z.c)

El proceso **Z** es la hoja encargada de desencadenar los eventos del sistema. Es el único proceso que hace uso de temporización asíncrona mediante señales del reloj del sistema.

- **Variables Importadas:**
    - extern pid_t pid_A: utilizado como destinatario de la señal de aviso tras la alarma.
    - extern pid_t pidSuperPadre: utilizado para la identificación por pantalla.
- **Llamadas al Sistema Utilizadas:** getpid(), getppid(), signal(), alarm(), pause(), kill(), exit().

Implementación del temporizador y señalización:

```
void codigo_Z(int segundos) {
    printf("Soy el proceso Z: mi pid es %d. Mi padre es %d. Mi
abuelo es %d. Mi bisabuelo es %d\n",
```

```
            getpid(), getppid(), pid_A, pidSuperPadre);

    // Configuración de la alarma asíncrona
    signal(SIGALRM, despertar);
    alarm(segundos);
    pause(); // Bloqueo de bajo consumo hasta la expiración

    // Despierta al expirar el tiempo y notifica a A
    kill(pid_A, SIGUSR1);

    // Queda a la espera de la fase final de destrucción
    signal(SIGUSR1, despertar);
    pause();

    printf("Soy Z (%d) y muero\n", getpid());
    exit(0);
}
```

3. Ejercicio 2 (hacha.c)

3.1. Enunciado del Ejercicio

Realizar un programa llamado **hacha.c** que divida un archivo en varios trozos con el mismo nombre y extensión h00, h01, … . Se debe tener en cuenta el formato y consideraciones siguientes:

$ hacha <archivo> <tamaño>

<archivo> nombre del archivo a dividir

<tamaño> tamaño en bytes de los archivos divididos

Para dividir, el proceso hacha (proceso padre) generará tantos procesos hijos como archivos se tengan que crear. Mediante tuberías, el proceso padre enviará la información a los hijos y serán estos últimos los que creen los archivos de destino y escriban en ellos.

$ hacha at_madrid.mp3 50000
$ ls
at_madrid.mp3.h00
at_madrid.mp3.h01
at_madrid.mp3.h02

Consideraciones

- Las entradas y salidas a los archivos se realizarán con llamadas al sistema estudiadas en la asignatura (no se puede utilizar printf, scanf, etc.)
- El padre creará tantos hijos como fragmentos del archivo a realizar, siendo decisión del alumno si los hijos se lanzan secuencial o concurrentemente.(En este caso los lanzamos de forma concurrente)

Ejemplo de Uso y Resultado Esperado

Sintaxis de ejecución:
$ ./hacha <archivo> <tamaño>

Ejemplo de salida en terminal y verificación en el sistema de ficheros:

```
$ ./hacha at_madrid.mp3 50000
$ ls
at_madrid.mp3  at_madrid.mp3.h00  at_madrid.mp3.h01  at_madrid.mp3.h02
```

3.2. Identificación y Justificación de Tareas

El análisis detallado de los requerimientos de la práctica permite estructurar el programa en las siguientes tareas clave:

3.2.1. Validación de Argumentos e Inspección del Fichero (struct stat)

Antes de iniciar cualquier reserva de recursos o creación de procesos, es indispensable validar que el usuario introduzca exactamente dos argumentos de línea de comandos (argc == 3). Posteriormente, se requiere inspeccionar el fichero de entrada para conocer su tamaño total en bytes. Para ello, se justifica el uso de la llamada al sistema stat(), la cual cumplimenta una estructura de datos del sistema Kernel del tipo struct stat. El campo st_size de dicha estructura proporciona de forma exacta el tamaño del archivo.

3.2.2. Cálculo de Fragmentación y Apertura en Modo Lectura

Conociendo el tamaño total y el tamaño de bloque solicitado por el usuario, la función calcular_num_trozos()determina el número total de fragmentos mediante una división entera, aplicando un redondeo hacia arriba si existe un resto no nulo. A continuación, el fichero de origen se abre mediante open() en modo solo lectura (O_RDONLY), obteniendo un descriptor de fichero (fd_in).

3.2.3. Creación Concurrente de Procesos y Tuberías Anónimas (IPC)

La distribución del fichero requiere crear tantos canales de comunicación independientes como fragmentos existan. Dentro de un bucle iterativo, el padre realiza las siguientes acciones por cada trozo:

- Invocación de pipe() para instanciar una tubería anónima unidireccional representada por el vector fd_pipe[2].
- Bifurcación del proceso mediante fork().
- **Gestión de descriptores en el hijo:** cierra el extremo de escritura fd_pipe[1] y asume el extremo de lectura fd_pipe[0].
- **Gestión de descriptores en el padre:** cierra el extremo de lectura fd_pipe[0] y mantiene el de escritura fd_pipe[1].

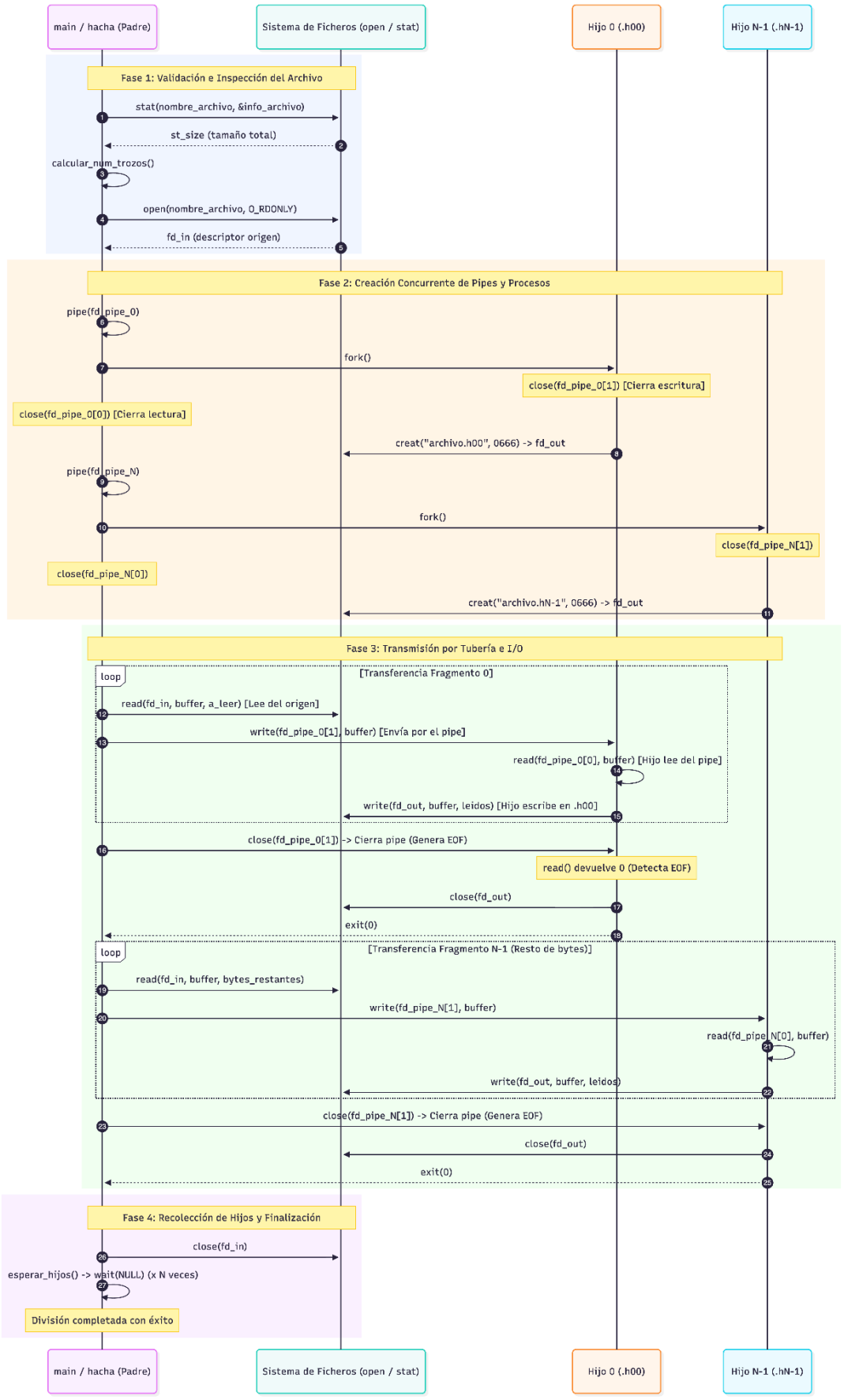
3.2.4. Transmisión de Datos por Tuberías y Apertura de Salidas (creat)

Cada proceso hijo genera su archivo destino asignando el sufijo correspondiente (.h00, .h01, ...) formateado mediante sprintf e invocando la llamada al sistema creat() con permisos de lectura/escritura (0666). El proceso padre lee bloques del fichero origen a través del descriptor compartido fd_in y los escribe en la tubería correspondiente. Dado que todos los hijos heredan el mismo descriptor fd_in, la posición del puntero de lectura interno del fichero se desplaza de forma secuencial y transparente entre las iteraciones.

3.2.5. Sincronización y Fin de Fichero (EOF)

Para señalar al hijo que la transmisión de su fragmento ha concluido, el padre cierra explícitamente su descriptor de escritura (close(fd_pipe_write)). Esto provoca que la llamada read() en el proceso hijo devuelva 0 (indicando EOF), desencadenando el cierre del archivo de salida y la finalización del hijo con exit(0). Finalmente, el padre ejecuta esperar_hijos(), que invoca wait(NULL) de forma repetida para recolectar a todos los procesos creados y prevenir la presencia de procesos zombi.**3.3 Diagrama de flujo**

3.3 Diagrama de flujo



3.4. Análisis Detallado del Código Fuente

Se analiza a continuación el diseño técnico de los bloques funcionales que forman hacha.c.(Nuevamente se ha omitido el manejo de errores menos importantes/directos para centrarnos en la interpretación teórica)

3.4.1. Módulo de Inspección, Apertura y Cálculo

Este bloque agrupa las funciones encargadas de interactuar con el sistema de ficheros para consultar metadatos y preparar los parámetros de la fragmentación.

- **Estructuras de Datos Empleadas:**
    - struct stat info_archivo: estructura definida en <sys/stat.h> utilizada por la llamada stat() para almacenar la información de estado del archivo. Se consulta el miembro info_archivo.st_size.
- **Llamadas al Sistema Utilizadas:** stat(), open().

Inspección del tamaño del archivo mediante struct stat:

```
int obtener_tamanyo_archivo(const char *nombre_archivo) {
    struct stat info_archivo;
    if (stat(nombre_archivo, &info_archivo) < 0) {
        perror("Error en stat");
        exit(1);
    }
    return info_archivo.st_size; // Devuelve el tamaño en bytes
}

int calcular_num_trozos(int tamanyo_total, int tamanyo_trozo) {
    int num_trozos = tamanyo_total / tamanyo_trozo;
    if (tamanyo_total % tamanyo_trozo != 0) {
        num_trozos++;
    }
    return num_trozos;
}
```

3.4.2. Lógica del Proceso Consumidor / Hijo (ejecutar_hijo)

Esta función encapsula el comportamiento de cada uno de los procesos hijos. Recibe como parámetro el descriptor de lectura de la tubería, el nombre base del archivo original y el índice de fragmento que le corresponde.

- **Comportamiento Técnico:** Formatea el nombre de salida (p. ej., fichero.bin.h02), abre un nuevo descriptor mediante creat() y se mantiene en un bucle de lectura/escritura en bloques de hasta 1024 bytes (char buffer[1024]). El bucle finaliza cuando read() detecta el cierre de la tubería por parte del padre.

- **Llamadas al Sistema Utilizadas:** creat(), read(), write(), close(), exit().

```
void ejecutar_hijo(int fd_pipe_read, const char *nombre_archivo,
int num_trozo) {
    char nombre_destino[256];
    sprintf(nombre_destino, "%s.h%02d", nombre_archivo,
num_trozo);

    // Creación del fichero de salida con permisos 0666
    int fd_out = creat(nombre_destino, 0666);
    if (fd_out < 0) {
        perror("Error en creat");
        exit(1);
    }

    // Volcado continuo desde la tubería hacia el fichero de
salida
    char buffer[1024];
    int leidos;
    while ((leidos = read(fd_pipe_read, buffer, sizeof(buffer))) >
0) {
        write(fd_out, buffer, leidos);
    }

    close(fd_out);
    close(fd_pipe_read);
    exit(0);
}
```

3.4.3. Lógica del Proceso Productor / Padre (ejecutar_padre)

La función ejecutar_padre() se encarga de transferir el volumen exacto de datos correspondiente a un fragmento específico hacia la tubería conectada con el hijo.

- **Control de Restos:** Calcula la cantidad exacta de bytes a enviar. Si se trata del último fragmento (num_trozo == num_trozos - 1), ajusta el límite de bytes al residuo resultante del módulo (tamanyo_total % tamanyo_trozo).

- **Inyección en la Tubería y EOF:** Utiliza un búfer temporal (tamaño limitado de transmisión por iteración)para leer del fichero origen (fd_in) y volcar en el descriptor de escritura de la tubería (fd_pipe_write). Al término de la transferencia del bloque, ejecuta close(fd_pipe_write) para enviar la señal implícita de fin de fichero al receptor.

- **Llamadas al Sistema Utilizadas:** read(), write(), close().

```
void ejecutar_padre(int fd_pipe_write, int fd_in, int num_trozo,
int num_trozos, int tamanyo_trozo, int tamanyo_total) {
    int bytes_a_enviar = tamanyo_trozo;
    if (num_trozo == num_trozos - 1 && (tamanyo_total %
tamanyo_trozo) != 0) {
        bytes_a_enviar = tamanyo_total % tamanyo_trozo;
    }

    char buffer[1024];
    int bytes_enviados = 0;

    while (bytes_enviados < bytes_a_enviar) {
        int a_leer = sizeof(buffer);
        if (bytes_a_enviar - bytes_enviados < sizeof(buffer)) {
            a_leer = bytes_a_enviar - bytes_enviados;
        }

        int leidos = read(fd_in, buffer, a_leer);
        if (leidos > 0) {
            write(fd_pipe_write, buffer, leidos);
            bytes_enviados += leidos;
        } else {
            break;
        }
    }
```

```
    // Cierre del extremo de escritura para notificar el fin de
datos al hijo
    close(fd_pipe_write);
}
```

3.4.4. Orquestación Concurrente y Sincronización Final

El bucle principal de creación de procesos (crear_fragmentos_concurrentes) centraliza la instanciación de las tuberías y la bifurcación de flujos.

- **Manejo de tuberías:** Para evitar bloqueos o fugas de recursos, se asegura el cierre inmediato del extremo no utilizado en cada rama tras el fork(): el hijo cierra fd_pipe[1] y el padre cierra fd_pipe[0].

- **Sincronización:** La función esperar_hijos() itera N veces ejecutando wait(NULL) para garantizar que el superpadre no retorne hasta que todas las tareas de escritura concurrentes en disco hayan finalizado con éxito.

- **Llamadas al Sistema Utilizadas:** pipe(), fork(), close(), wait().

```
void crear_fragmentos_concurrentes(int fd_in, const char
*nombre_archivo, int num_trozos, int tamanyo_trozo, int
tamanyo_total) {
    for (int i = 0; i < num_trozos; i++) {
        int fd_pipe[2];

        if (pipe(fd_pipe) < 0) {
            perror("Error en pipe");
            exit(1);
        }

        pid_t pid = fork();

        if (pid < 0) {
            perror("Error en fork");
            exit(1);
        }
        else if (pid == 0) {
            close(fd_pipe[1]); // El hijo no escribe en la tubería
            ejecutar_hijo(fd_pipe[0], nombre_archivo, i);
        }
        else {
```

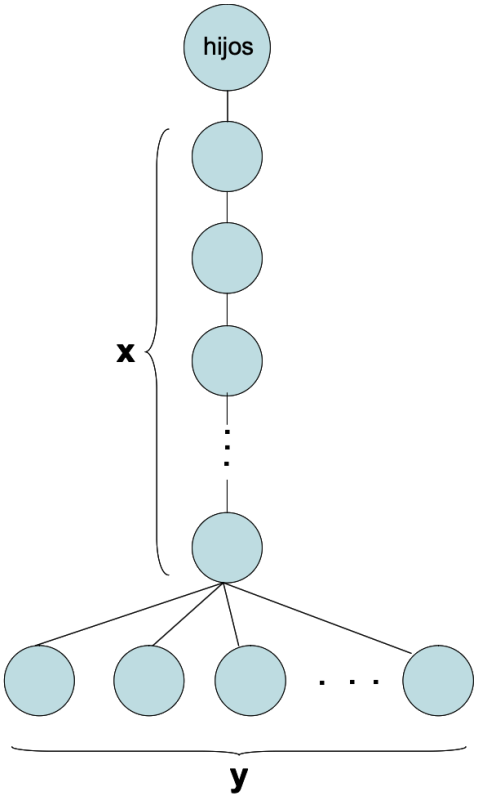
```
            close(fd_pipe[0]); // El padre no lee de la tubería
            ejecutar_padre(fd_pipe[1], fd_in, i, num_trozos,
tamanyo_trozo, tamanyo_total);
        }
    }
}

void esperar_hijos(int num_hijos) {
    for (int i = 0; i < num_hijos; i++) {
        wait(NULL);
    }
}
```

4. Ejercicio 3 (hijos.c)

4.1. Enunciado del Ejercicio

Diseña un programa llamado **hijos.c** que cree un árbol de procesos según la siguiente estructura a partir de dos parámetros
$hijos x y



El proceso **hijos** debe mostrar el siguiente mensaje:
“Soy el superpadre (pid) : mis hijos finales son: pidx1, pidx2, pidx3, …, pidxy”
Los procesos de la parte baja deben mostrar el siguiente mensaje**:**
“Soy el subhijo pidx1, mi padres son: pid1, pid2, …, pidx”

Ejemplo de salida por terminal(./hijos 3 4)

Soy el subhijo 44822, mi padres son: 44819, 44820, 44821

Soy el subhijo 44823, mi padres son: 44819, 44820, 44821

Soy el subhijo 44824, mi padres son: 44819, 44820, 44821

Soy el subhijo 44825, mi padres son: 44819, 44820, 44821

Soy el superpadre (44819) : mis hijos finales son: 44822, 44823, 44824, 44825

4.2. Identificación y Justificación de Tareas

A partir de los requisitos del enunciado, se estructuran las siguientes tareas de diseño técnico y modularización:

4.2.1. Validación de Parámetros de Entrada (parse)

Se comprueba mediante la función parse() que se introduzcan exactamente dos argumentos adicionales (argc == 3), verificando mediante validaciones aritméticas que sean enteros estrictamente positivos y evitando valores nulos que invaliden el diseño.

4.2.2. Gestión Centralizada de Memoria Compartida (struct MemoriaCompartida)

Dado que el superpadre y los subhijos finales residen en ramas separadas, la visibilidad de los PIDs se resuelve mediante un segmento de memoria compartida privada. Para encapsular su gestión, se independizan las tareas de inicialización y liberación mediante funciones dedicadas como crearMemoriaCompartida() y liberarMemoriaCompartida(), operando sobre la estructura:

```
typedef struct {
    int x;                  // Longitud de la cadena vertical
    int y;                  // Número de subhijos horizontales
    pid_t padres[100];      // PIDs de los nodos de la cadena
vertical
    pid_t subhijos[100];    // PIDs de los subhijos horizontales
} MemoriaCompartida;
```

4.2.3. Creación Recursiva de la Cadena Vertical (crearVertical)

En lugar de un bucle iterativo, crearVertical() adopta una estructura recursiva basada en el parámetro nivel_actual. Para mantener una alta legibilidad y evitar código complejo (*espagueti*), la lógica se desglosa en funciones auxiliares como

esUltimoNodo(), ejecutarHojaVertical() y propagarTerminacionYMorir(). Cada nivel registra su propio PID en la celda correspondiente de mem->padres[nivel_actual].

4.2.4. Generación Horizontal y Modularización de Subhijos (crearHorizontal)

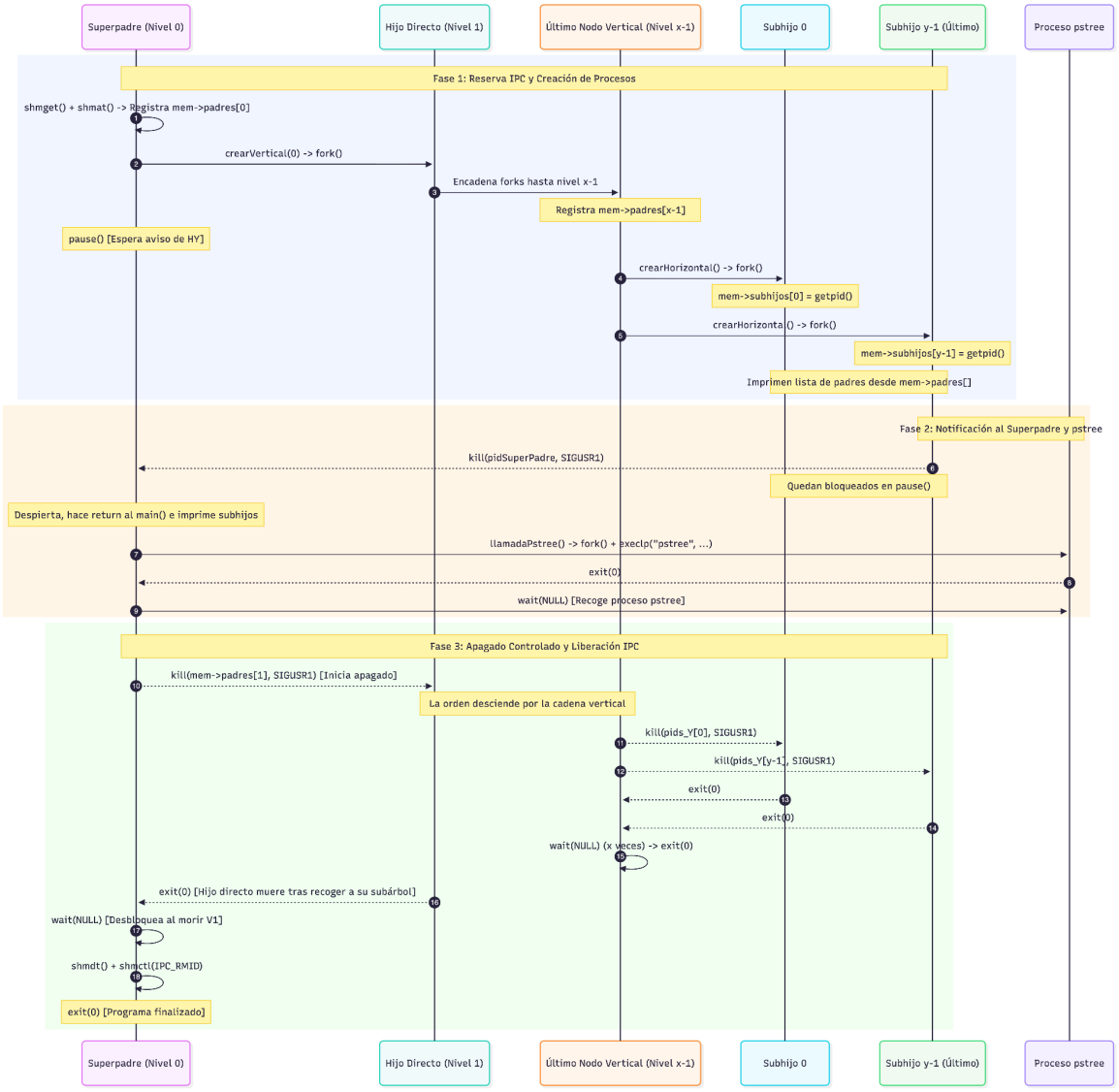
Al alcanzar el último nivel vertical, se invoca crearHorizontal() para engendrar a los y subhijos mediante un bucle de bifurcaciones. Las tareas individuales de cada hijo se han modularizado en funciones limpias:

- registrarSubhijo(): guarda el PID en el array compartido.
- imprimirPadresSubhijo(): lee y muestra por pantalla la cadena completa de ascendientes.
- terminarSubhijo(): bloquea al hijo en pause() y asegura que el último elemento de la fila ($col = y - 1$) notifique al superpadre enviando SIGUSR1.

4.2.5. Ejecución de pstree y Destrucción en Cascada

Al recibir la notificación, el superpadre reanuda su ejecución retornando al main(), imprime los PIDs de los subhijos finales mediante imprimirSubhijosFinales() e invoca un proceso efímero con llamadaPstree() para mostrar la jerarquía. A continuación, emite SIGUSR1 al primer hijo vertical (mem->padres[1]), propagando el apagado de arriba hacia abajo y recolectando las muertes mediante wait(), para finalmente desvincular (shmdt) y destruir (shmctl con IPC_RMID) el segmento de memoria compartida.

4.3 Diagrama de flujo



4.4. Análisis Detallado del Código Fuente

4.4.1. Módulo Principal y Gestión IPC (main)

Este bloque coordina la inicialización del programa, la reserva del segmento IPC y la orquestación del flujo principal tras el despertar del superpadre.

- **Variables Globales:** pidSuperPadre, shmid y el puntero global mem.
- **Llamadas al Sistema Utilizadas:** getpid(), shmget(), shmat(), fork(), execlp(), wait(), kill(), shmdt(), shmctl(), exit().

```
int main(int argc, char *argv[]){
    parse(argc, argv);
    configurarSenyales();
    pidSuperPadre = getpid();

    int x = atoi(argv[1]);
    int y = atoi(argv[2]);

    // Declaramos e inicializamos la memoria compartida
    mem = crearMemoriaCompartida(x, y);

    // Inicia la cadena vertical recursiva
    crearVertical(0, x, y);

    // Al despertar el SuperPadre y retornar al main, imprime el
mensaje final
    imprimirSubhijosFinales(y);

    // Ejecutamos el pstree para mostrar la jerarquía
    llamadaPstree(getpid());
    wait(NULL);

    // Si x > 1(es decir, es el superPadre), despierta a la cadena
vertical para iniciar la muerte en cascada
    if (x > 1) {
        kill(mem->padres[1], SIGUSR1);
        wait(NULL);
    }

    // Liberamos recursos del segmento de memoria compartida
    liberarMemoriaCompartida();
    exit(0);
}
```

4.4.2. Módulo de Creación Vertical Recursiva (crearVertical)

Gestiona la profundización de la columna y la transición hacia la capa horizontal mediante funciones auxiliares.

- **Comportamiento Técnico:** Utiliza esUltimoNodo() para comprobar si se ha alcanzado la hoja vertical. Si es así, ejecuta ejecutarHojaVertical(); en caso contrario, bifurca el siguiente nivel con fork() y delega la propagación de

señales en propagarTerminacionYMorir().

- **Llamadas al Sistema Utilizadas:** getpid(), fork(), pause(), kill(), wait(), exit().

```
int esUltimoNodo(int nivel_actual, int x) {
    return nivel_actual == x - 1;
}

void terminarHijosHorizontal(pid_t pids_Y[], int y) {
    for(int j = 0; j < y; j++){
        kill(pids_Y[j], SIGUSR1);
    }
    for(int j = 0; j < y; j++){
        wait(NULL);
    }
}

void ejecutarHojaVertical(int nivel_actual, int y) {
    pid_t pids_Y[y];
    crearHorizontal(y, pids_Y);
    pause();
    terminarHijosHorizontal(pids_Y, y);

    if (nivel_actual == 0) return; // Si x == 1, el SuperPadre
retorna al main
    exit(0);
}

void propagarTerminacionYMorir(pid_t pidHijo) {
    kill(pidHijo, SIGUSR1);
    wait(NULL);
    exit(0);
}

void crearVertical(int nivel_actual, int x, int y) {
    mem->padres[nivel_actual] = getpid();

    if (esUltimoNodo(nivel_actual, x)) {
        ejecutarHojaVertical(nivel_actual, y);
        return;
    }
```

```
    pid_t pid = fork();

    if (pid == 0) {
        crearVertical(nivel_actual + 1, x, y);
        exit(0);
    } else if (pid > 0) {
        configurarSenyales();
        pause();

        if (nivel_actual > 0) {
            propagarTerminacionYMorir(pid);
        }
        return; // Retorno al main() en el superpadre
(nivel_actual == 0)
    } else {
        perror("Error en fork\n");
        exit(1);
    }
}
```

4.4.3. Módulo de Subhijos Horizontales (crearHorizontal)

Instancia la fila final de y procesos y delega sus responsabilidades en funciones auxiliares dedicadas.

- **Comportamiento Técnico:** Cada hijo registrado mediante registrarSubhijo() imprime su ascendencia con imprimirPadresSubhijo() y queda bloqueado mediante terminarSubhijo(). El último subhijo emite la señal SIGUSR1 hacia el superpadre.

- **Llamadas al Sistema Utilizadas:** fork(), getpid(), kill(), pause(), exit().

```
void registrarSubhijo(int indice) {
    mem->subhijos[indice] = getpid();
}

void imprimirPadresSubhijo() {
    printf("Soy el subhijo %d, mi padres son:", getpid());
    for (int k = 0; k < mem->x; k++) {
        printf(k == mem->x - 1 ? " %d" : " %d,", mem->padres[k]);
```

```
    }
    printf("\n");
}

void terminarSubhijo(int indice, int y) {
    if (indice == y - 1) {
        kill(pidSuperPadre, SIGUSR1);
    }
    pause();
    exit(0);
}

void crearHorizontal(int y, pid_t pids_Y[]){
    for(int i = 0; i < y; i++){
        pid_t pid = fork();
        if(pid == 0){
            configurarSenyales();
            registrarSubhijo(i);
            imprimirPadresSubhijo();
            terminarSubhijo(i, y);
        } else {
            pids_Y[i] = pid;
        }
    }
}
```

5. Recursos utilizados

5.1 Mermaid.live

Herramienta gratuita utilizada para diseñar los diagramas de flujo de una manera sencilla para mi inexperiencia general con diagramas de flujo y que a pesar demuestra resultados estéticos

5.2 Github

Enlace al repositorio con el código fuente de la práctica completa documentado mediante readmes y código comentado que permite comprender la práctica a nivel puramente funcional (sin embargo no la perspectiva teórica)sin necesidad de leer la memoria completa. También incluye códigos de prueba que se utilizaron para

aprender y comprender mejor los conceptos utilizados para todos los ejercicios de la práctica.
https://github.com/toribiocuevasdaniel/SO/tree/main/P1